

La solution : Model Context Protocol (MCP)

Un adaptateur universel pour outils

Référence : [Documentation ADK - Outils MCP](#) | [Documentation MCP officielle](#)

Qu'est-ce que MCP ?

MCP (Model Context Protocol) est une **norme ouverte** créée par Anthropic et désormais hébergée par la Linux Foundation. Elle permet aux agents IA de se connecter à des serveurs d'outils externes par le biais d'un protocole universel. C'est un peu comme une clé USB pour les outils IA : n'importe quel serveur compatible avec MCP peut fournir des outils à n'importe quel client compatible avec MCP, quel que soit le framework d'IA utilisé.

Extrait de la documentation ADK :

"ADK vous aide à la fois à utiliser et à exploiter les outils MCP dans vos agents. Consultez la documentation sur les outils MCP pour obtenir des exemples de code et des modèles de conception qui vous aideront à utiliser ADK avec les serveurs MCP."

Deux modèles d'intégration avec ADK :

ADK prend en charge deux modèles d'intégration MCP :

1. **Utiliser des serveurs MCP existants** : votre agent ADK agit en tant que client MCP. Il exploite les outils fournis par des serveurs externes (*ce module*).
2. **Exposer des outils ADK via MCP** : vous créez un serveur MCP qui encapsule vos outils ADK, les rendant accessibles à tout client MCP (*modèle avancé, voir la documentation*).

Ce module se concentre sur le **premier modèle** : se connecter à des serveurs MCP préexistants pour étendre les capacités de votre agent.

L'importance du MCP

Le pouvoir de la normalisation :

Sans MCP	Avec MCP
Code d'intégration personnalisé pour chaque outil	Un seul modèle d'intégration pour tous les outils
Outils liés à des frameworks d'IA spécifiques	Outils fonctionnant avec Claude, GPT, Gemini, etc.
Vous gérez vous-même chaque intégration	Maintenance par la communauté et les

Sans MCP	Avec MCP
	fournisseurs
Limité à ce qui est intégré à votre framework	Accès à un écosystème toujours plus vaste comprenant des centaines d'outils

Adoption par l'industrie :

-  **Anthropic** a créé le protocole (novembre 2024).
-  **OpenAI** a officiellement adopté MCP (mars 2025).
-  **Google ADK** offre une prise en charge native de MCP avec **McpToolset**.
-  La **Linux Foundation** héberge désormais MCP en tant que norme ouverte.

Quel impact pour vous ?

- **Écrivez une fois, utilisez partout** : les outils MCP fonctionnent avec les différents frameworks d'IA.
- **Exploitez l'écosystème** : utilisez des outils conçus par des spécialistes (bases de données, API, services cloud).
- **Restez à jour** : dès que de nouveaux serveurs MCP sont publiés, votre agent peut immédiatement les utiliser.
- **Séparez les tâches** : concentrez-vous sur la logique de votre agent et laissez les responsables de la maintenance des outils gérer les intégrations.

Recherche de serveurs MCP

Avant de développer des outils personnalisés, vérifiez s'il existe déjà un serveur MCP pour votre cas d'utilisation.

Ressources officielles :

Ressource	Description	Lien
Registre de serveurs MCP	Catalogue officiel consultable des serveurs MCP	registry.modelcontextprotocol.io
Serveurs de référence	Serveurs officiels gérés par l'équipe MCP	github.com/modelcontextprotocol/servers
Documentation MCP	Spécifications complètes du protocole et guides	modelcontextprotocol.io

Les serveurs MCP les plus populaires incluent les éléments suivants :

- **Système de fichiers** : lecture/écriture de fichiers et de répertoires
- **GitHub** : gestion des dépôts, problèmes, demandes d'extraction

- **Slack** : envoi de messages, lecture de canaux
- **PostgreSQL/MySQL** : requêtes de base de données
- **Notion** : accès aux documents et aux bases de données
- **Google Drive** : accès au stockage de fichiers

Conseil : Recherchez dans le [registre de serveurs MCP](#) avant d'écrire des outils personnalisés. La capacité dont vous avez besoin existe peut-être déjà.

Comment fonctionne MCP dans ADK

Flux d'intégration :

```
graph LR
  A[Agent ADK] --> B[McpToolset]
  B --> C[Serveur MCP]
  C --> D[Outils : list_directory, read_file, etc.]

  style A fill:#e1f5ff
  style B fill:#ffeb99
  style C fill:#eefbee
```

Processus :

1. Les **serveurs MCP** exposent des outils par le biais du protocole MCP standardisé.
2. Les **agents ADK** se connectent aux serveurs à l'aide de **McpToolset**.
3. **Les outils sont détectés automatiquement.** Aucune inscription manuelle n'est requise.
4. **Votre agent utilise ces outils** exactement comme n'importe quel autre outil ADK.

*Les agents ADK se connectent aux serveurs MCP avec **McpToolset**, de façon à détecter automatiquement les outils disponibles.*

Concepts fondamentaux

1. La classe **McpToolset**

Référence : [Documentation ADK - Outils MCP](#)

Extrait de la documentation ADK :

"La classe **McpToolset** peut être directement ajoutée à la liste **tools** de votre agent, ce qui permet de se connecter facilement à un serveur MCP, de détecter ses outils et de les mettre à disposition de votre agent."

Que fait **McpToolset ?**

1. **McpToolset se connecte** à un serveur MCP en utilisant les paramètres de connexion.

2. McpToolset **détecte** automatiquement les outils disponibles sur le serveur.
3. McpToolset **relaie** les appels d'outils de votre agent vers le serveur MCP.
4. McpToolset **renvoie** les résultats à votre agent.

Utilisation :

```
from google.adk.agents import LlmAgent
from google.adk.tools.mcp_tool import McpToolset
from google.adk.tools.mcp_tool.mcp_session_manager import StdioConnectionParams
from mcp import StdioServerParameters

agent = LlmAgent(
    model='gemini-2.5-flash',
    name='my_agent',
    instruction="Utilise les outils disponibles pour aider les utilisateurs.",
    tools=[
        McpToolset(
            connection_params=StdioConnectionParams(
                server_params=StdioServerParameters(
                    command='npx',
                    args=['-y', '@some/mcp-server'],
                ),
            ),
        ),
    ],
)
```

Points à noter :

- **McpToolset** s'ajoute à la liste **tools** comme n'importe quel autre outil.
- Les paramètres de connexion indiquent à ADK comment accéder au serveur MCP.
- Les outils du serveur deviennent automatiquement accessibles à votre agent.

2. Types de connexions

Référence : [Documentation ADK - Outils MCP](#)

ADK prend en charge deux méthodes de connexion aux serveurs MCP :

StdioConnectionParams (serveurs locaux)

Lance un serveur MCP en tant que sous-processus local sur votre machine.

```
from google.adk.tools.mcp_tool.mcp_session_manager import StdioConnectionParams
from mcp import StdioServerParameters

# Connexion au serveur local
connection = StdioConnectionParams(
    server_params=StdioServerParameters(
        command='npx',
        # Commande à exécuter
```

```

        args=['-y', '@modelcontextprotocol/server-filesystem', '/path'], # Arguments
    ),
)

```

Quand l'utiliser : Développement, tests, scénarios impliquant un seul utilisateur.

SseConnectionParams (serveurs distants)

Se connecte à un serveur MCP fonctionnant à distance via HTTP.

```

from google.adk.tools.mcp_tool.mcp_session_manager import SseConnectionParams

# Connexion au serveur distant
connection = SseConnectionParams(
    url="https://your-mcp-server.example.com/sse",
    headers={'Authorization': 'Bearer YOUR_TOKEN'},
)

```

Quand l'utiliser : Déploiements de production, serveurs MCP hébergés dans le cloud.

Comparaison des types de connexion :

Aspect	StdioConnectionParams	SseConnectionParams
Emplacement du serveur	Local (sous-processus)	Distant (HTTP)
Configuration	Simple (commande npx)	Nécessite une infrastructure de serveurs
Application idéale	Développement, tests	Production, cloud
Évolutivité	Utilisateur unique	Multi-utilisateurs

3. Filtrage des outils

Référence : [Documentation ADK - Outils MCP](#)

Les serveurs MCP peuvent exposer de nombreux outils, mais vous préférerez peut-être que votre agent n'en utilise que certains. Le paramètre `tool_filter` vous permet de contrôler les outils disponibles.

Pourquoi filtrer les outils ?

-  **Sécurité** : limiter aux outils en lecture seule, exclure les opérations dangereuses
-  **Simplicité** : ne donner à l'agent que ce dont il a besoin pour accomplir sa tâche
-  **Concentration** : réduire le risque de confusion découlant d'un trop grand nombre d'outils disponibles

Utilisation :

```
McpToolset(  
    connection_params=StdioConnectionParams(...),  
    tool_filter=['list_directory', 'read_file'], # Uniquement ces outils  
)
```

Exemple : Restreindre l'accès au système de fichiers

 Bonne pratique : n'exposer que des outils sûrs et en lecture seule

```
McpToolset(  
    connection_params=StdioConnectionParams(  
        server_params=StdioServerParameters(  
            command='npx',  
            args=['-y', '@modelcontextprotocol/server-file-system', '/allowed/path'],  
        ),  
    ),  
    tool_filter=['list_directory', 'read_file'], # Pas d'accès en écriture  
)
```

 Pratique risquée : exposer tous les outils, y compris les opérations d'écriture

```
McpToolset(  
    connection_params=StdioConnectionParams(  
        server_params=StdioServerParameters(  
            command='npx',  
            args=['-y', '@modelcontextprotocol/server-file-system', '/'],  
        ),  
    ),  
    # Aucun filtre = tous les outils sont exposés, y compris write_file, delete, etc.  
)
```

Bonne pratique : Utilisez toujours `tool_filter` en production pour n'exposer que les outils dont votre agent a besoin.

4. Choisir le bon type d'outil

Quand faut-il utiliser des outils MCP plutôt que des outils intégrés ou des outils de fonction personnalisés ?

Le cadre décisionnel :

Avez-vous besoin de cette fonctionnalité ?

- └─> Est-ce une recherche Google ou une exécution de code ?
 - └─> OUI → Utilisez les outils intégrés (module 2).
- └─> Existe-t-il déjà un serveur MCP pour cela ?
 - └─> OUI → Utilisez les outils MCP (ce module).
 - Consultez : registry.modelcontextprotocol.io
- └─> S'agit-il de votre propre logique métier ou d'un système propriétaire ?
 - └─> OUI → Écrivez des outils de fonction personnalisés (module 4).

Principales différences :

Aspect	Outils MCP	Outils intégrés	Outils de fonction personnalisés
Responsable(s) de la maintenance	Communauté/fournisseurs	Équipe ADK	Vous
Effort de configuration	Connecter et configurer	Importer	Écriture de code
Portabilité	Fonctionnent avec les différents frameworks d'IA	ADK uniquement	ADK uniquement
Personnalisation	Limitée aux options du serveur	Limitée	Contrôle complet
Durée d'implémentation	Minutes	Minutes	Heures/Jours

Les outils MCP sont idéaux dans les situations suivantes :

-  La fonctionnalité est **courante** (fichiers, bases de données, API populaires).
-  Quelqu'un d'autre a **déjà résolu** le problème d'intégration.
-  Vous êtes à la recherche de **portabilité** entre les différents frameworks d'IA.
-  Vous préférez utiliser des **outils gérés** plutôt que d'écrire vos propres outils.

Les outils de fonction personnalisés sont plus adaptés dans les situations suivantes :

-  La logique est **unique** à votre entreprise.
-  Vous vous connectez à des systèmes internes **propriétaires**.
-  Vous avez besoin d'un **contrôle total** sur la gestion des comportements et des exceptions.
-  Il n'existe **aucun serveur MCP** pour votre cas d'utilisation spécifique.

Exemple concret de prise de décisions :

Besoin	Décision	Explication
Lire des fichiers depuis un disque	MCP : <code>@modelcontextprotocol/server-filesystem</code>	Accéder au système de fichiers est une fonctionnalité courante : il existe un serveur MCP.
Interroger une base de données PostgreSQL	MCP : <code>@modelcontextprotocol/server-postgres</code>	Accéder aux bases de données est une fonctionnalité courante : il existe un serveur MCP.
Calculer des frais de port	Outil de fonction personnalisé	Il s'agit d'une logique propre à

Besoin	Décision	Explication
		vosre activité : vous seul connaissez vos tarifs.
Accéder à l'API interne des employés	Outil de fonction personnalisé	Il s'agit d'un système propriétaire, pas de serveur MCP public.

Exemple pratique

Créer un assistant de lecture de fichiers avec des outils MCP

Objectif : Créer un agent capable de lister et de lire des fichiers à l'aide du serveur MCP de système de fichiers.

Prérequis :

- Node.js et npm sont installés (pour la commande npx).

Étape 1 : Créer le projet

```
adk create file_reader_assistant
cd file_reader_assistant
```

Fonction :

- Crée un nouveau répertoire de projet ADK
- Établit la structure d'agent de base
- Crée `agent.py` avec le code d'agent par défaut

Étape 2 : Créer un dossier de test contenant des fichiers

```
mkdir -p my_files
echo "Bonjour, ceci est un fichier de test." > my_files/hello.txt
echo "Ceci est un autre fichier dont le contenu est différent." > my_files/notes.txt
```

Fonction :

- Crée un dossier auquel le serveur MCP aura accès
 - Ajoute des exemples de fichiers à lire par l'agent
-

Étape 3 : Écrire l'agent

Remplacez le contenu de `agent.py` par :

```
"""
Agent assistant de lecture de fichiers
Démontre l'intégration des outils MCP avec ADK à l'aide du serveur MCP de système de
fichiers.

Référence : https://google.github.io/adk-docs/tools-custom/mcp-tools/
"""

import os
from google.adk.agents import LlmAgent
from google.adk.tools.mcp_tool import McpToolset
from google.adk.tools.mcp_tool.mcp_session_manager import StdioConnectionParams
from mcp import StdioServerParameters

# Définir le dossier auquel autoriser l'accès aux fichiers (il doit s'agir d'un chemin
absolu)
ALLOWED_PATH = os.path.abspath("./my_files")

# Créer le dossier s'il n'existe pas
os.makedirs(ALLOWED_PATH, exist_ok=True)

# Créer l'agent avec les outils de système de fichiers MCP
root_agent = LlmAgent(
    model='gemini-2.5-flash',
    name='file_reader_assistant',
    description='Aide les utilisateurs à lire et à explorer des fichiers à l'aide des
outils MCP.',
    instruction="""
    Tu es un assistant de lecture de fichiers qui aide les utilisateurs à explorer des
fichiers.

    Tes capacités :
    - Lister les fichiers dans les répertoires à l'aide de list_directory
    - Lire le contenu des fichiers à l'aide de read_file

    Lorsque tu aides les utilisateurs, procède comme suit :
    1. Utilise list_directory pour afficher les fichiers disponibles.
    2. Utilise read_file pour afficher le contenu d'un fichier lorsqu'on te le
demande.
    3. Décris ce que tu trouves de manière utile.

    Veille toujours à bien identifier le dossier sur lequel tu travailles.
    """,
    tools=[
        McpToolset(
            connection_params=StdioConnectionParams(
                server_params=StdioServerParameters(
                    command='npx',
                    args=[
```

```

        '-y',
        '@modelcontextprotocol/server-filesystem',
        ALLOWED_PATH,
    ],
),
),
# Filtrer pour n'exposer que les outils sûrs en lecture seule
tool_filter=['list_directory', 'read_file'],
)
],
)

```

Tutoriel du code :

Lignes 9 à 14 : installation et configuration

- Importez `McpToolset` et les classes de connexion depuis ADK.
- Importez `StdioServerParameters` depuis le package MCP.
- Définissez `ALLOWED_PATH` comme un chemin absolu vers le dossier des fichiers.

Lignes 17 et 18 : créer un dossier

- Assurez-vous que le dossier cible existe avant l'exécution de l'agent.

Lignes 21 à 47 : créer un agent avec les outils MCP

- `McpToolset` se connecte au serveur MCP de système de fichiers.
- `StdioConnectionParams` lance le serveur avec npx.
- `tool_filter` limite les outils à `list_directory` et `read_file`.
- Les instructions guident l'agent sur la manière d'utiliser ces outils.

Étape 4 : Exécuter et tester

adk web

Accédez à <http://localhost:8000> dans votre navigateur.

Étape 5 : Scénarios de test

Test 1 : Lister les fichiers

Vous : `Quels fichiers se trouvent dans le dossier ?`

Comportement attendu :

- L'agent appelle l'outil `list_directory` par le biais de MCP.

- L'outil renvoie la liste des fichiers du dossier `my_files`.
- L'agent vous présente la liste des fichiers.

Points à noter :

- L'agent a automatiquement détecté et utilisé l'outil MCP.
 - Aucun code d'outil personnalisé n'a été nécessaire : le serveur MCP a tout fourni.
-

Test 2 : Lire un fichier

Vous : `Montre-moi le contenu de hello.txt`

Comportement attendu :

- L'agent appelle l'outil `read_file` avec le chemin du fichier.
- L'outil renvoie le contenu du fichier.
- L'agent vous présente le contenu.

Points à noter :

- L'agent a correctement appelé l'outil MCP avec le bon argument.
 - Le contenu du fichier est affiché via le protocole MCP.
-

Test 3 : Opérations multiples

Vous : `Liste les fichiers, puis lis le fichier notes.txt`

Comportement attendu :

- L'agent appelle d'abord `list_directory`.
- L'agent appelle ensuite `read_file` pour notes.txt.
- L'agent présente les deux résultats simultanément.

Points à noter :

- L'agent orchestre plusieurs appels d'outils MCP.
 - Les modèles que vous avez appris dans le module 1 s'appliquent aux outils MCP.
-

Points à retenir

En quoi consiste MCP et pourquoi est-ce important ?

- **Norme ouverte** : les agents IA peuvent se connecter à des serveurs d'outils externes.

- **Interopérabilité universelle** : fonctionne avec les différents frameworks d'IA (ADK, Claude, GPT).
- **Adoption par l'industrie** : Anthropic, OpenAI, Google et la Linux Foundation.
- **Accès à l'écosystème** : des centaines d'outils prédéfinis pour les tâches courantes.

Utilisation de MCP dans ADK :

- **McpToolset** connecte votre agent aux serveurs MCP.
- **Les outils sont détectés automatiquement**, sans inscription manuelle.
- **Utilisez-le comme n'importe quel autre outil** : les modèles du module 1 s'appliquent.

Types de connexions :

- **StudioConnectionParams** - Serveurs locaux par le biais d'un sous-processus (développement)
- **SseConnectionParams** - Serveurs distants par le biais de HTTP (production)

Sécurité avec filtrage des outils :

- Utilisez **tool_filter** pour n'exposer que les outils nécessaires.
- Si possible, **restez en lecture seule** pour renforcer la sécurité.
- **Moins d'outils = décisions plus claires** pour votre agent.

Quand utiliser chaque type d'outil ?

Scénario	Méthode	Pourquoi
Capacité préexistante (fichiers, bases de données, API)	Outils MCP	Maintenance par la communauté
Capacités Google (recherche, exécution de code)	Outils intégrés	Maintenance par l'équipe ADK
Logique spécifique à votre entreprise	Outils de fonction personnalisés	Vous seul connaissez votre entreprise

Principales ressources :

- [Registre des serveurs MCP](#) pour trouver les serveurs MCP existants
- [Documentation MCP officielle](#) : documentation du protocole
- [Guide MCP ADK](#) : documentation sur l'intégration ADK

Au-delà de ce module : capacités MCP avancées

Ce module a abordé les principes fondamentaux de l'utilisation des serveurs MCP existants. MCP et ADK offrent des capacités supplémentaires pour des cas d'utilisation plus avancés.

Que pouvez-vous faire d'autre avec MCP dans ADK ?

Capacité	Description	Documentation
Créer votre propre serveur MCP	Intégrez vos outils personnalisés dans un serveur MCP pour les partager avec d'autres personnes.	Documentation ADK - Créer des serveurs MCP
Exposer les outils ADK via MCP	Rendez vos outils ADK accessibles à n'importe quel client MCP (Claude, GPT, etc.).	Documentation ADK - Outils MCP
Se connecter à plusieurs serveurs MCP	Utilisez les outils de plusieurs serveurs MCP dans un seul agent.	Documentation ADK - Outils MCP
Déployer des serveurs MCP sur Cloud Run	Hébergez vos serveurs MCP pour une utilisation en production.	Documentation ADK - Outils MCP

Le deuxième modèle d'intégration (avancé) :

Comme nous l'avons mentionné précédemment, ADK est compatible avec deux modèles d'intégration MCP. Le deuxième modèle (**exposer les outils ADK via un serveur MCP**) vous offre plusieurs possibilités :

1. Intégrer vos outils ADK existants dans un serveur MCP
2. Mettre ces outils à la disposition de tout client compatible avec MCP
3. Partager vos outils avec l'écosystème de l'IA dans son ensemble

C'est utile lorsque vous souhaitez que d'autres frameworks d'IA (Claude, GPT, etc.) utilisent les outils que vous avez développés dans ADK.